The default behavior is for the application to fetch the public keys lazily upon the first received HTTP request rather than at startup, both for performance and coupling reasons (you don't need Keycloak up and running when starting the application). OAuth2 Authorization Servers provide their public keys using the JSON Web Key (JWK) format. The collection of public keys is called a *JWK Set*. The endpoint where Keycloak exposes its public keys is called the *JWK Set URI*. Spring Security will automatically rotate the public keys whenever Keycloak makes new ones available.

For each incoming request containing an Access Token in the `Authorization` header, Spring Security will automatically validate the token's signature using the public keys provided by Keycloak and decode its claims via a `JwtDecoder` object, which is auto-configured behind the scenes.

In the Catalog Service project (catalog-service), open the application.yml file, and add the following configuration.

Listing 12.7 Configuring Catalog Service as an OAuth2 Resource Server

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:                                          ❶
          issuer-uri:
➥http://localhost:8080/realms/PolarBookshop          ❷
```

❶ OAuth2 doesn't enforce a data format for Access Tokens, so we must be explicit about our choice. In this case, we want to use JWT.

❷ The Keycloak URL providing information about all the relevant OAuth2 endpoints for the specific realm

NOTE Explaining the cryptographic algorithm used to sign the Access Token is out of scope for this book. If you'd like to learn more about cryptography, you might want to look up *Real-World Cryptography* by David Wong (Manning, 2021).